

```
self.scoreText.draw()  
self.messageText.draw()
```

Listing 12-4: The `Game` object that runs the game

In the `__init__()` method ❶, we initialize a number of instance variables that only need to be set up once. We create the `Deck` object, set the starting score, and create a `DisplayText` object for displaying the score and the result of each move. We also load a number of sound files for use during play. Lastly, we call the `reset()` method ❷, which contains any code needed for one play of the game: that is, to shuffle the deck, play the shuffling sound, deal out eight cards, display them in previously computed positions, and show the face of the first card.

When the user presses the Higher or Lower button, the main code calls `hitHigherOrLower()` ❸, which turns over the next card, compares the value with the previous face-up card, and awards or subtracts points.

The `draw()` method ❹ iterates through all the cards in the current game, telling each to draw itself (by calling each `Card` object's `draw()` method). It then draws the text of the score and the feedback for the current move.

Testing with `__name__`

When you write a class, it's always a good idea to write some test code to ensure that an object created from that class will work correctly. As a reminder, any file containing Python code is called a *module*. A standard practice is to write one or more classes in a module, then use an `import` statement to bring that module into some other module. When you write a module that contains a class (or classes), you can add some test code that's intended to run *only* when the module is run as the main program, and doesn't run in the typical case when the module is imported by another Python file.

In a project with multiple Python modules, you typically have one main module and several other modules. When your program runs, Python creates the special variable `__name__` in every module. In whichever module is given control first, Python sets the value of `__name__` to the string `'__main__'`.